



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ingeniería

Compilers
Lexer

Student: Cortés Guillén Pavel Francisco - 118001568
Cojulum Ortega Mabel Ximena - 321101921
Gómez Meza Jorge Ángel - 118003225
Monroy Tapia Jesus Alejandro - 321045979
Villa Garcia Arely Guadalupe - 321000697

Professor: René Adrián Dávila Pérez.

Group: 5

Semester: 2027-1

Deadline: October 6th, 2026

1 Introduction

The compilation process is fundamentally divided into several phases, with lexical analysis acting as the critical first step. This project presents the design and implementation of a hand-crafted Lexical Analyzer (Lexer) developed in Python. The primary objective of this module is to read raw source code, discard irrelevant characters such as whitespace and comments, and tokenize the input sequence into meaningful symbols.

Unlike automated solutions that rely on tools like FLEX or standard regular expression libraries, this lexer is built entirely from scratch using deterministic finite automaton (DFA) state functions. It is capable of recognizing 40 distinct token types (including keywords, identifiers, operators, constants, and punctuators) while distinguishing case sensitivity. Furthermore, the lexer implements a robust “Panic Mode” error recovery strategy, allowing it to detect and report multiple lexical errors—such as malformed numbers or unclosed string literals—without halting execution. Ultimately, the lexer produces a clean stream of tokens that forms the foundation for subsequent syntax analysis.

2 Architecture and Project Structure

The architectural design of the lexer follows a modular, layered approach, ensuring separation of concerns and maintainability. The package, structured under `unam.fi.compilers.g5.02`, relies on five core components that coordinate the data flow from raw text to formatted token reports.

- **Main CLI (`main.py`):** Acts as the orchestrator and entry point for the application. It utilizes `argparse` to handle command-line inputs (strings or files) and directs the overall execution flow.
- **Reader (`io/reader.py`):** Responsible for input mechanism abstraction. It safely loads the source code from either direct string inputs or UTF-8 encoded files.
- **Scanner (`lexer/scanner.py`):** The core engine that iterates through the input character by character. It coordinates the DFA states, manages the current line and column pointers, and registers any encountered errors using a Panic Mode recovery mechanism.
- **DFA and Symbol Table (`lexer/dfa.py` & `symbol_table/symtab.py`):** The DFA module implements the State Pattern, using independent functions for each state to identify identifiers, numbers, strings, and operators via a “maximal munch” strategy. The Symbol Table applies early insertion; once a valid word is identified, it is queried to determine if it is a boolean literal, a reserved keyword, or a standard identifier.
- **Writer (`io/writer.py`):** The final output layer that processes the list of valid tokens and lexical errors to generate a structured, human-readable report.

The data flows linearly: raw characters enter the Scanner, which interacts with the DFA and Symbol Table to extract lexemes. These are converted into immutable Token objects and passed to the Writer for final presentation.

3 Theoretical Foundation

Although the primary scope of this project is the construction of a Lexical Analyzer, its true purpose is to supply a valid stream of tokens to a Parser (Syntax Analyzer). A parser evaluates these tokens against a Context-Free Grammar (CFG) to build a parsing tree, representing the syntactic structure of the program through a series of derivations and tree expansions.

To achieve efficient and deterministic parsing, top-down parsers (such as LL(1) predictive parsers) require grammar formulations that are free of both left recursion and ambiguity. If a grammar exhibits left recursion, a top-down parser can fall into an infinite loop. Similarly, if multiple production rules share common prefixes, the parser cannot determine which derivation to apply based on a single lookahead token. Therefore, grammar transformations like Left Factoring are strictly necessary to guarantee the veracity and performance of the parser relying on our lexer.

3.1 Context-Free Grammar (CFG) Proposal

The following CFG excerpts demonstrate the theoretical syntax for variable declarations and print statements within our custom language. The terminal symbols in this grammar directly correspond to the token categories emitted by our Lexer (e.g., `INT`, `ID`, `ASSIGN`, `INTEGER`, `SEMICOLON`).

The initial, unfactored grammar proposal for variable declarations and `printf` statements is defined as follows:

$\text{Stmt} \rightarrow \text{int id} = \text{num} ;$
 $\text{Stmt} \rightarrow \text{int id} ;$
 $\text{Stmt} \rightarrow \text{printf} (\text{string}) ;$
 $\text{Stmt} \rightarrow \text{printf} (\text{string} , \text{ArgList}) ;$

3.2 Left Factoring Application

In the grammar proposed above, ambiguity arises for an LL(1) parser. Upon reading the `int` token, the parser cannot decide whether it is dealing with a variable assignment or a simple variable declaration without looking further ahead than one token. The same issue occurs with the `printf` token, as both productions begin with the common prefix `printf ( string`.

To resolve this and make the grammar LL(1) compliant, we apply Left Factoring. This process extracts the common prefixes and postpones the parsing decision to a newly generated non-terminal symbol.

Factored Declaration Grammar: $\text{Stmt} \rightarrow \text{int id } \textit{Rest}$
 $\textit{Rest} \rightarrow = \text{num} ; | ;$

Factored Print Function Grammar: $\text{Stmt} \rightarrow \text{printf} (\text{string } \textit{PrintRest}$
 $\textit{PrintRest} \rightarrow) ; | , \text{ArgList}) ;$
 $\textit{ArgList} \rightarrow \text{id } \textit{ArgList}'$
 $\textit{ArgList}' \rightarrow , \text{id } \textit{ArgList}' | \epsilon$

By transforming the grammar using left factoring, we guarantee that every parsing step is deterministic. Our Lexer bridges the gap by strictly enforcing token boundaries (maximal munch) and handling invalid characters beforehand, allowing the LL(1) parser to seamlessly consume the input and perform error-free derivations.

4 Lexer Implementation

The core implementation of the lexical analyzer resolves character recognition deterministically without relying on regular expression engines or third-party parsing tools[cite: 1, 12, 30]. Instead, the scanner relies on a 13-state Deterministic Finite Automaton (DFA) defined formally as $M = (Q, \Sigma, \delta, q_0, F)$ [cite: 5, 6, 19]:

- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}, q_{11}, q_{12}\}$ [cite: 5, 6].
- $\Sigma = L \cup D \cup \{ws, \#, =, !, <, >, +, -, *, /, (,), \{, \}, ;, ,, ., "\} \cup \Sigma_{other}$ [cite: 5].
- $\delta : Q \times \Sigma \rightarrow Q$ is the state transition function implemented directly in Python[cite: 1, 5, 11].
- q_0 is the start state[cite: 5, 19].
- $F = \{q_1, q_2, q_4, q_6, q_7, q_8, q_9, q_{10}, q_{12}\}$ is the set of valid accepting states[cite: 5, 6, 19].

Multi-character lexemes are processed using the *Maximal Munch* (longest match) strategy[cite: 6, 19]. When the automaton enters an accepting state, scanning continues as long as character inputs yield valid state transitions[cite: 6, 19].

4.1 Input Mechanism

The input subsystem is encapsulated within `io/reader.py`. The application provides two main entry functions: `read_string(source)` and `read_file(filepath)`. When reading from disk, the file is loaded into memory as a UTF-8 string buffer.

The `Scanner` class (`lexer/scanner.py`) controls character evaluation through three main state pointers:

1. **pointer**: Absolute 0-based index referencing the current character position in the buffer.
2. **line**: 1-based index incremented whenever a newline character (`\n`) is encountered.
3. **column**: 1-based character position relative to the current line origin.

Listing 1 demonstrates the main iteration loop implemented within `scanner.py`.

Listing 1: Core scanning loop in `lexer/scanner.py`.

```
class Scanner:
    def __init__(self, source_code: str):
        self.source = source_code
        self.pointer = 0
        self.line = 1
        self.column = 1
        self.tokens = []
        self.errors = []

    def scan_all(self):
        while self.pointer < len(self.source):
            char = self.source[self.pointer]
```

```

# Skip whitespace and update positional coordinates
if char.isspace():
    if char == '\n':
        self.line += 1
        self.column = 1
    else:
        self.column += 1
    self.pointer += 1
    continue

# Discard single-line comments starting with '#'
if char == '#':
    self.consume_comment()
    continue

# Invoke state handler according to the initial character
if char.isalpha() or char == '_':
    self.process_identifier_or_keyword()
elif char.isdigit():
    self.process_numeric_literal()
elif char == '"':
    self.process_string_literal()
else:
    self.process_operator_or_punctuation()

```

4.2 Token Classification

The lexer recognizes 40 internal token types. To meet output formatting requirements, the writer layer (`io/writer.py`) maps these 40 internal types into 5 human-readable categories:

- **keyword**: Reserved language constructs (e.g., `printf`, `print`, `int`, `if`, `else`).
- **identifier**: User-defined variable and function names.
- **operator**: Mathematical, logical, and relational operators (`=`, `==`, `!=`, `<`, `<=`, `>`, `>=`, `+`, `-`, `*`, `/`).
- **constant**: Integer numbers, floating-point numbers, booleans, and string literals.
- **punctuation**: Structural symbols (`(`, `)`, `{`, `}`, `;`, `,`).

Table 1 details the complete state transition matrix governing the automaton.

4.2.1 Symbol Table Integration

Alphanumeric lexemes ending in state q_1 are passed to `symbol_table/symtab.py`:

1. Check for boolean literals (`True`, `False`). If matched, return `BOOL_LITERAL`.

Table 1: State Transition Matrix for the Lexer DFA ($q_0 \dots q_{12}$).

State	Input Character	Next State	Action / Token Emitted
q_0	Letter / _	q_1	Begin identifier or keyword scan
q_0	Digit	q_2	Begin integer literal scan
q_0	"	q_5	Begin string literal scan
q_0	#	q_{11}	Single-line comment (discarded)
q_0	=, !, <, >	q_7, q_8, q_9, q_{10}	Multi-character operator evaluation
q_0	+, -, *, /, (,), {, }, :, ,	q_{12}	Emit single-character operator / punctuation
q_0	Whitespace	q_0	Advance coordinates; skip
q_0	Unrecognized char	—	Trigger Panic Mode Recovery
q_1	Alphanumeric / _	q_1	Continue word accumulation
q_1	Non-alphanumeric	q_{12}	SymbolTable lookup → Emit Keyword or ID
q_2	Digit	q_2	Continue digit accumulation
q_2	.	q_3	Transition to fractional part
q_2	Non-digit / Non-.	q_{12}	Emit INTEGER
q_3	Digit	q_4	Accumulate required fractional digit
q_3	Non-digit	—	Error: Malformed real number (e.g., 3.)
q_4	Digit	q_4	Continue fractional digit accumulation
q_4	Non-digit	q_{12}	Emit REAL_NUMBER
q_5	"	q_6	Complete string literal
q_5	Char (not " or \n)	q_5	Accumulate string body
q_5	EOF / \n	—	Error: Unclosed string literal
q_6	Any	q_{12}	Emit STRING_LITERAL
q_7	=	q_{12}	Emit EQ (==)
q_7	Other	q_{12}	Emit ASSIGN (=)
q_8	=	q_{12}	Emit NEQ (!=)
q_8	Other	—	Error: Standalone invalid operator !
q_9	=	q_{12}	Emit LEQ (<=)
q_9	Other	q_{12}	Emit LT (<)
q_{10}	=	q_{12}	Emit GEQ (>=)
q_{10}	Other	q_{12}	Emit GT (>)
q_{11}	\n	q_0	End comment; resume scanning
q_{11}	Non-\n	q_{11}	Discard comment content
q_{12}	—	q_0	Reset state for next token

2. Perform a case-sensitive lookup against the 15 reserved keywords (e.g., `printf`, `print`, `int`). If matched, emit the corresponding keyword token.
3. Otherwise, insert the word into the `SymbolTable` and assign token type ID.

4.2.2 Panic Mode Error Recovery

When encountering illegal characters (e.g., `@`) or unexpected transitions (e.g., malformed float `3.`), Panic Mode records the error message, line, and column index in `scanner.errors`. It discards the invalid character and continues scanning from the next position without halting.

5 Test Cases and Results

The lexer was verified using unit tests and the mandatory reading benchmark:

```
printf("This is an example"); int a = 10;
```

Scanning this input produces 10 tokens and 0 lexical errors. Table 2 presents the token classification and coordinate breakdown generated by `io/writer.py`.

Table 2: Token breakdown for the mandatory test code.

No.	Internal Token	Category	Lexeme	Line	Column
1	PRINTF	keyword	'printf'	1	1
2	L_PAREN	punctuation	'('	1	7
3	STRING_LITERAL	constant	'"This is an example"'	1	8
4	R_PAREN	punctuation	')'	1	28
5	SEMICOLON	punctuation	';'	1	29
6	INT	keyword	'int'	1	31
7	ID	identifier	'a'	1	35
8	ASSIGN	operator	'='	1	37
9	INTEGER	constant	'10'	1	39
10	SEMICOLON	punctuation	';'	1	41

Formatted Console Output:

```
- ANALIZADOR LEXICO -
```

```
keyword punctuation constant punctuation punctuation keyword identifier operator const
```

```
Total de tokens: 10
```

```
ERRORES LEXICOS: Sin errores lexicos.
```

5.1 Extended Test Cases and Edge Cases

To confirm robust tokenization and Panic Mode recovery across all 40 token types, additional test cases were executed (Table 3).

In Test Case 15 (`int a=3.; b=! @ $`), Panic-Mode recovery isolates 4 distinct errors (malformed float `3.`, standalone operator `!`, illegal symbols `@` and `$`) while returning all 7 surrounding valid tokens[cite: 3, 8, 21].

Table 3: Evaluation of edge cases and Panic Mode recovery.

ID	Input Snippet	Output Stream Categories	Tokens	Error Status[cite: 1, 2, 20]
1	<code>print</code>	<code>keyword</code>	1	None[cite: 8]
2	<code>Print</code>	<code>identifier</code> (Case-sensitive)	1	None[cite: 1, 8]
3	<code>printf</code>	<code>keyword</code> (PRINTF vs PRINT)	1	None[cite: 8, 28]
4	<code>printed</code>	<code>identifier</code> (Longest match)	1	None[cite: 1, 8]
5	<code>10-3</code>	<code>constant operator constant</code>	3	Int + MINUS + Int
6	<code>"a" + "b"</code>	<code>constant operator constant</code>	3	Valid string literal
7	<code>a <= b</code>	<code>identifier operator identifier</code>	3	Relational operator
8	<code>x == 3.14</code>	<code>identifier operator constant</code>	3	Floating-point real
9	<code>a = 1; # comment</code>	<code>identifier operator constant punctuation</code>	4	Comment ignored
10	<code>@</code>	None	0	Error: Illegal character
11	<code>"abc</code>	None	0	Error: Unclosed string
12	<code>3.</code>	None	0	Error: Malformed number
13	<code>1abc</code>	<code>constant identifier</code>	2	Split: 1 and abc[cite: 1, 2, 20]
14	<code>!</code>	None	0	Error: Standalone punctuation
15	<code>int a=3.; b=! @ \$</code>	7 Tokens Emitted	7	4 Panic Errors Logged

6 Conclusion

This report details the implementation of a custom lexical analyzer designed under package namespace `unam.fi.compilers.g5.02`[cite: 1, 14, 30]. By avoiding automatic toolchains such as FLEX or built-in regular expression engines, the project demonstrates a full implementation of theoretical compiler concepts[cite: 1, 12, 30].

The lexer executes a 13-state Deterministic Finite Automaton ($q_0 \dots q_{12}$) structured through modular state functions[cite: 4, 5, 11]. Core technical achievements include:

- **Comprehensive Classification:** Full recognition of 40 internal token types mapped to 5 user-facing categories[cite: 1, 2, 20].
- **Fault Resilience:** Implementation of Panic-Mode error recovery, enabling the scanner to log character coordinates for malformed lexemes without interrupting execution[cite: 3, 11, 21].
- **Decoupled Modular Design:** Strict division between stream reading (`Reader`), state scanning (`Scanner/DFA`), symbol tracking (`SymbolTable`), output formatting (`Writer`), and web presentation (`Flask`)[cite: 4, 16, 17].

The resulting token stream and populated `SymbolTable` provide a solid foundation for integration with the upcoming *LL*(1) predictive syntax parser[cite: 7, 11, 26].